CIS 4020 · DATA STRUCTURES

Trees and Binary Search Trees

Terminology, binary tree shapes, and the binary search tree, with Java and C/C++ examples

---

I think that I shall never see
A poem lovely as a tree.

Joyce Kilmer, "Trees," 1913

A note before you begin. A word before you read. These are my notes from years of teaching this unit, cleaned up so they are readable. While C/C++ was my first language, I am a bit rusty because I have taught Java for years now. For these notes, I have been using AI to help type these up, check for issues, and fill in explanations, which means a wrong or imprecise statement can slip in and survive longer than it should, and I do not fully trust the AI, so I am still editing. They are not meant to be a study guide, they are not gospel, and not everything here shows up in a lecture (and some lecture items are not here). If you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, tell me. You will be doing students and me a favor, and you will understand it better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. We are a two-language course, thus I will do my best to show everything in Java (CIS) and C/C++ (CS).

1. Where trees fit

Every ADT we have built so far has been **linear**: each item has at most one item before it and one item after it.

- **Lists, stacks, and queues** are linear and **position oriented**. You ask for an item by where it sits: the item at position 3, the top of the stack, the front of the queue.
- **Sorted lists** are linear and **value oriented**. Items are placed by their value, but you still walk the list one item at a time to find one.
- **Trees** are **nonlinear** and value oriented. An item can have several items directly below it, which is what lets a well shaped tree find an item without looking at most of the others.

A tree is still an ADT. The program asks to add, remove, or search, and what happens on the other side of the wall is hidden. There are many varieties of trees, and this course only touches some of them:

- General trees
- Binary trees and binary search trees *(this unit)*
- Balanced trees: AVL, red-black, 2-3, 2-3-4, B-tree *(the next unit)*
- Others: splay, trie, suffix, Huffman, heap, and more

2. Tree terminology

A tree is a set of **nodes** connected by **edges**, with one node at the top and no cycles: there is exactly one path from the top to any node. Computer scientists draw trees upside down, with the root at the top and the leaves at the bottom. Figure 1 is the tree used for every example in Table 1.

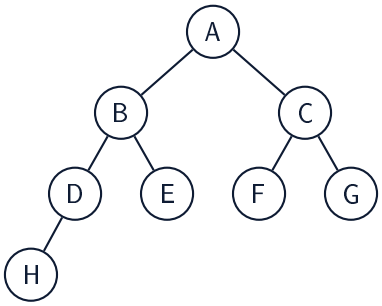


Figure 1: A binary tree with eight nodes. A is the root. A has children B and C. B has children D and E. C has children F and G. D has one child, H, on its left. E, F, G, and H are leaves.

Table 1: Tree terms, each with an example from Figure 1.

Term	Meaning	In Figure 1
Node	One element of the tree; it holds an item and links to its children.	A through H
Edge	The connection between a parent and one of its children. In code, an edge is a reference (Java) or a pointer (C++).	A to B, B to D, D to H, and so on (eight nodes, seven edges)
Root	The one node with no parent, at the top.	A
Leaf	A node with no children.	E, F, G, H
Interior node	A node that is not a leaf (it has at least one child).	A, B, C, D
Parent	The node directly above a node, joined by an edge.	B is the parent of D and E
Child	A node directly below a node, joined by an edge.	D and E are children of B
Sibling	Nodes that share the same parent.	D and E; F and G
Ancestor	Any node on the path from the root down to a node, not counting the node itself.	The ancestors of H are D, B, A
Descendant	Any node below a node, on a path that leads away from the root.	The descendants of B are D, E, H
Path	A sequence of nodes where each pair in a row is joined by an edge.	A, B, D, H
Subtree	A node together with all of its descendants.	B, D, E, H form the subtree rooted at B

Term	Meaning	In Figure 1
Level	The root is at level 1; a child is one level below its parent.	A: 1; B, C: 2; D to G: 3; H: 4
Height	The number of nodes on the longest path from the root to a leaf. An empty tree has height 0.	4 (the path A, B, D, H)

Watch the height convention. These notes, and our textbook, count *nodes* for height, so a single node has height 1 and the tree in Figure 1 has height 4. Many other books and websites count *edges* instead, which makes every height one smaller (a single node has height 0, and Figure 1 has height 3). Neither is wrong; just know which one a source is using before comparing numbers. Some books also leave the root out of "interior node."

3. General trees and binary trees

In a **general tree**, each node (except the root) has one parent and may have **zero or more** children. A file system is a general tree: a folder can hold any number of folders. In a **binary tree**, each node has one parent and **zero, one, or two** children, and the two child positions are named: a **left child** and a **right child**. A node with only one child still says which side it is on. Figure 1 is a binary tree, and the fact that H is D's *left* child is part of the tree's shape.

Each child of a node is the root of a smaller tree, so we talk about a node's **left subtree** and **right subtree**. This is why recursion fits trees so well: nearly every tree algorithm does something at a node and then handles each subtree as a smaller version of the same problem. An empty subtree is the base case.

Full and complete binary trees

Two shapes come up often enough to have names. Let h be the height of the tree.

- A **full binary tree** has no "missing" nodes. Every leaf is at level h, and every node that is not a leaf has exactly two children.
- A **complete binary tree** is full down to level $h - 1$, and level h is filled in from left to right with no gaps (it does not have to be filled all the way).

Every full binary tree is also complete, but not the other way around.



Figure 2: Left: a full binary tree of height 3, with seven nodes; every level is filled. Right: a complete binary tree of height 4; levels 1 to 3 are full, and level 4 has three nodes packed in from the left.

A full binary tree of height h has exactly $2^h - 1$ nodes, because level k holds 2^{k-1} nodes and $1+2+4+\cdots+2^{h-1} = 2^h - 1$. Turned around, that gives the most useful fact in this unit: a binary tree with n nodes has height

$$\lceil \log_2(n+1) \rceil \;\le\; h \;\le\; n.$$

The lower bound is a tree packed as tightly as possible (full or complete). The upper bound is a tree where every node has only one child, which is really a linked list drawn at an angle.

Balanced and non-balanced trees

A binary tree is **height balanced**, or just **balanced**, when, at *every* node, the heights of the left and right subtrees differ by at most one. A balanced tree stays close to the minimum height for its number of nodes. If the heights differ by more than one at any node, the tree is **not balanced**.

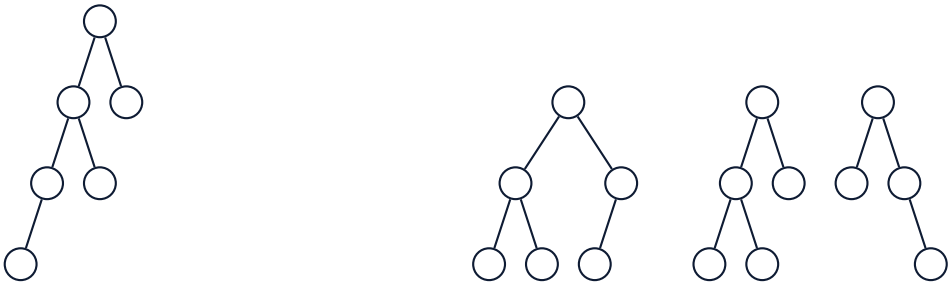


Figure 3: Left: not balanced. At the root, the left subtree has height 3 and the right subtree has height 1, a difference of two. Right: three balanced trees. In the first, both subtrees of the root have height 2; in the second, they have heights 2 and 1. In the third, the root's left child is a leaf and its right child has only a right child, so the subtree heights are 1 and 2. That third tree is balanced but not complete, because its bottom level has a gap on the left. In all three, every other node also passes the test.

How do these shapes relate? Every full binary tree is complete, and every complete binary tree is balanced: the levels above the bottom are full, and the bottom level fills from the left, so no node can have one subtree two levels taller than the other. None of these arrows reverses, though. A balanced tree can have gaps anywhere, as long as the subtree heights at every node differ by at most one. The third balanced tree in Figure 3 is an example, and the balanced trees in the next unit are usually balanced without being complete.

Checking only the root is not enough. A tree can have equal heights on both sides of the root and still be badly lopsided further down. Keeping a tree balanced as items are added and removed is the whole subject of the next unit.

4. Representing a binary tree in code

Most of the time we use a **reference-based** (C/C++: pointer-based) representation, the same idea as the nodes in a linked list. Two array-based forms come at the end of this section. Each node holds an item and two links, one to the left child and one to the right child. A link that leads nowhere is `null` in Java and `nullptr` in C++. The tree object itself keeps one link, `root`.

Java: Node.java

```
class Node<T> {
    T item;
    Node<T> left;
    Node<T> right;
```

```
    public Node(T item) {
        this.item = item;
        this.left  = null;
        this.right = null;
    }
}
```

C++: a node, nested inside the tree class

```
struct Node {
    ItemType item;
    Node* left;
    Node* right;
    explicit Node(const ItemType& it) : item(it), left(nullptr), right(nullptr) {}
};
```

You can reach any node by following links from the root. In Figure 1, if `root` refers to A, then `root.left` is B, `root.right` is C, and `root.left.left` is D (in C++: `root->left->left`).

A general tree cannot use a fixed pair of links, because a node may have any number of children. A common choice is to give each node an **array (or list) of child references** in place of `left` and `right`. Binary trees are what we use for the rest of this unit.

Array-based representations

A binary tree can also live in an array, with no references at all. There are two common ways to do it, and both are shown here with the tree in Figure 1.

Version A: an array of nodes with child indices. Each array slot holds an item plus the *index* of its left child and its right child, with -1 meaning "no child." This is exactly the reference-based tree, except that an index plays the role of a reference. The tree also keeps the index of its root (here, 0). Table 2 stores the nodes in breadth-first order, but any order works, because the child indices say where everything is.

Table 2: Figure 1 stored as an array of nodes with child indices.

Index	Item	Left child index	Right child index
0	A	1	2
1	B	3	4
2	C	5	6
3	D	7	-1
4	E	-1	-1
5	F	-1	-1
6	G	-1	-1
7	H	-1	-1

When a node is removed, its slot becomes free. The tree keeps a **free list**, a chain of unused slots

linked through their child index field, so that the next insertion can reuse a freed slot instead of growing the array.

Version B: position-based, with no links. Here the position of an item *is* its place in the tree. The root goes in index 0, and for the node at index i:

$$\text{left child} = 2i + 1, \qquad \text{right child} = 2i + 2, \qquad \text{parent} = \lfloor (i - 1)/2 \rfloor \text{ (integer division)}.$$

The formulas are the same in Java and C++. Some books start the root at index 1 instead, which changes the formulas to $2i$, $2i + 1$, and $\lfloor i/2 \rfloor$.

Table 3: Figure 1 stored as a position-based array.

Index	0	1	2	3	4	5	6	7
Item	A	B	C	D	E	F	G	H

Check one: D is at index 3, so its left child is at $2(3) + 1 = 7$, which is exactly where H is, and H's parent is at $\lfloor (7 - 1)/2 \rfloor = 3$. Figure 1 happens to be a complete tree, so every slot from 0 to 7 is used.

Version B stays compact only for a complete tree. For any other shape, the slots where nodes are missing still take up room, because a node's position is fixed by its path from the root. In the worst case, a tree of height h needs all $2^h - 1$ slots (the count from Section 3) no matter how few nodes it has. The seven keys inserted in sorted order in Figure 5 form a chain of height 7, so storing them this way needs 127 slots for 7 items. That is why Version B is used for trees that are complete by design. The **heap**, from the list of tree varieties, is the best known example. Version A works for any shape, at the cost of storing two indices with every item.

5. The binary search tree

A **binary search tree** (BST) is a binary tree that keeps its items in order by a **search key**. For *every* node in the tree:

- every key in the node's **left** subtree is **less than** the node's key, and
- every key in the node's **right** subtree is **greater than** the node's key.

Because each subtree of a BST is itself a BST, this is a recursive definition, and the algorithms that follow are recursive for the same reason.

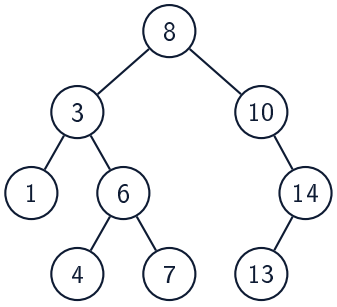


Figure 4: A binary search tree with root 8. The left subtree (3, 1, 6, 4, 7) holds only keys less than 8, and the right subtree (10, 14, 13) holds only keys greater than 8. The same rule holds at node 3, node 6, node 10, and node 14.

Note that the rule is about the whole subtree, not just the children. In Figure 4, 7 is in the left subtree of 8, so it must be less than 8, even though its parent is 6.

Items with keys

A BST stores whole items, such as a student record, but compares only their keys, such as a student ID. In our Java code, an item class extends `KeyedItem`, which holds the key and provides `getKey()`. The key type must be `Comparable`, so the tree can call `compareTo`. The key has no setter: if the key of an item already in the tree could change, the item could end up on the wrong side of its parent.

Java: KeyedItem.java

```
public abstract class KeyedItem<KT extends Comparable<? super KT>> {

    private KT key;

    public KeyedItem(KT key) {
        this.key = key;
    }

    public KT getKey() {
        return this.key;
    }
}
```

Java: an item class and the tree in use

```
public class Student extends KeyedItem<Integer> {
    private String name;

    public Student(int id, String name) {
        super(id);           // the ID is the search key
        this.name = name;
    }

    public String getName() { return name; }
}

// elsewhere
BST<Student, Integer> roster = new BST<>();
roster.add(new Student(60, "Ada"));
roster.add(new Student(20, "Grace"));
Student s = roster.search(20);   // finds Grace by key alone
roster.remove(60);
```

In C++ the same job is done by a template: the item type only has to provide a `getKey()` member function, and the key type has to support `<` and `==`.

C++: an item type and the tree in use

```
struct Student {
    int id;
```

```
    std::string name;
    int getKey() const { return id; }   // the ID is the search key
};

BST<Student, int> roster;
roster.add(Student{60, "Ada"});
roster.add(Student{20, "Grace"});
const Student& s = roster.search(20);   // finds Grace by key alone
roster.remove(60);
```

Searching

To search for a key, start at the root and compare:

- If the subtree is empty, the key is not in the tree.
- If the key equals the node's key, you found it.
- If the key is less, search the left subtree. If it is greater, search the right subtree.

Each comparison throws away an entire subtree. Searching Figure 4 for 7 visits only 8, 3, 6, and 7. Searching for 5 visits 8 (go left), 3 (go right), 6 (go left), and 4 (go right), and the right subtree of 4 is empty, so 5 is not there. That empty spot is exactly where 5 would be added, which is the idea behind insertion.

Inserting

To insert, search for the new item's key. The search ends at an empty subtree, and that is where the new node goes. A new item is **always added as a leaf**; nothing already in the tree moves.

The Java method returns the root of the subtree it was given, and the caller stores the result back into the link it followed (`tn.left = insert(tn.left, item)`). That is how the new node gets attached to its parent: when the recursion reaches `null`, it returns the new node, and the parent stores it.

Table 4 traces inserting 60, 20, 70, 10, 40, 30, 50 into an empty tree. The result is the tree in Figure 5, on the left.

Table 4: Inserting 60, 20, 70, 10, 40, 30, 50, in that order, into an empty BST.

Insert	Keys compared along the way	Placed as
60	(tree is empty)	the root
20	60 (less, go left)	left child of 60
70	60 (greater, go right)	right child of 60
10	60 (left), 20 (left)	left child of 20
40	60 (left), 20 (right)	right child of 20
30	60 (left), 20 (right), 40 (left)	left child of 40
50	60 (left), 20 (right), 40 (right)	right child of 40

The order of insertion decides the shape

The same seven keys, inserted in a different order, build a different tree. Figure 5 shows three orders.

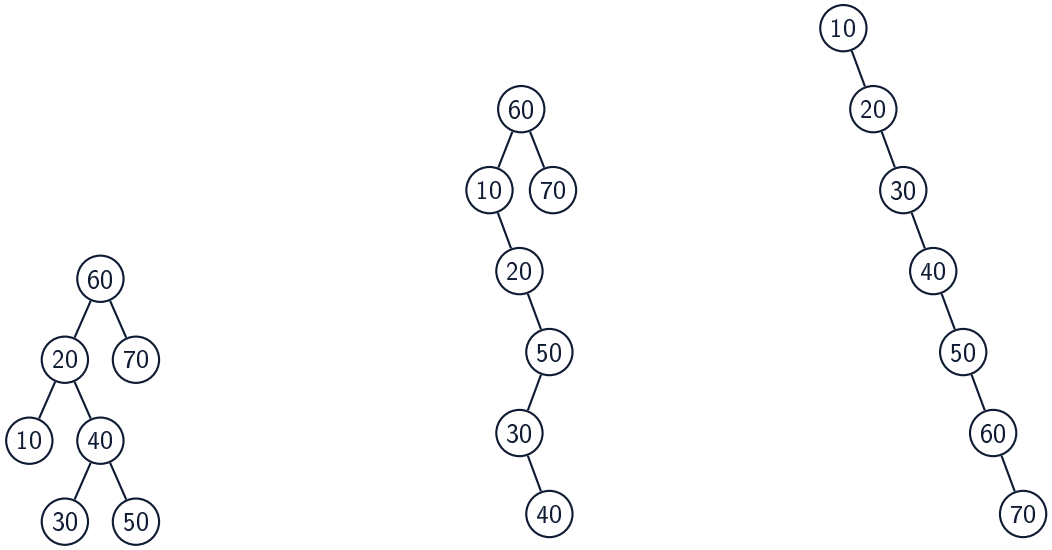


Figure 5: Three BSTs holding the keys 10 to 70. Left: inserted as 60, 20, 70, 10, 40, 30, 50, height 4. Middle: inserted as 60, 10, 70, 20, 50, 30, 40, height 6; under 10 the tree runs 20, 50, 30, 40, zigzagging right, left, right. Right: inserted in sorted order 10, 20, 30, 40, 50, 60, 70, height 7; every node has only a right child.

Inserting keys in sorted order is the worst thing you can do to a plain BST: every new key is the largest so far, so it goes to the right of every node, and the tree turns into a linked list. Sorted input is common in real programs (think of IDs handed out in increasing order), which is a big reason balanced trees exist.

Deleting

Deleting starts with a search for the key. If the search reaches an empty subtree, the key is not in the tree. Otherwise, let `tn` be the node to delete, `p` its parent, and `c` its child (when it has exactly one). What happens next depends on how many children `tn` has. The examples use the tree in Figure 6.

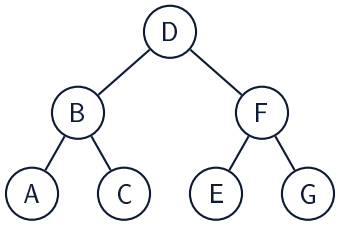


Figure 6: The starting tree for the deletion cases: D is the root, with children B and F. B has children A and C, and F has children E and G. Letters are the keys, in alphabetical order.

Case 1: `tn` is a leaf. Remove it by setting the parent's link to null.

```
if (tn is the left child of p)   p.left  = null
else                             p.right = null
```

Deleting E from Figure 6 sets `F.left` to null; deleting G sets `F.right` to null.

Case 2: `tn` has one child `c`. Let the parent adopt the child: the child moves up and takes `tn`'s place.

```
if (tn is the left child of p)   p.left  = c
else                             p.right = c
```

This keeps the ordering, because all of `c`'s subtree was already on the correct side of `p`.



Figure 7: Case 2. Before: F (highlighted with a double border) has only a left child, E. After deleting F, E becomes the right child of D.

The same works for the other side. If B had only the child C, deleting B would make C the left child of D.

Case 3: `tn` has two children. The parent cannot adopt two children into one link, so we do not remove the *node* at all. Instead:

```
find M, the inorder successor of tn
copy M's item into tn
remove M from the tree
```

The **inorder successor** of `tn` is the node with the next larger key: the **leftmost node of `tn`'s right subtree**. To find it, go right once, then go left until you cannot go left any more. Because M is the smallest key on the right side, putting it in `tn`'s place keeps everything on the left smaller and everything on the right larger. And because M has no left child (otherwise it would not be the leftmost), **removing M is always case 1 or case 2**.



Figure 8: Case 3, deleting the root D (highlighted with a double border). Go right once to F, then left as far as possible, which reaches E. E is the inorder successor. Copy E into the root, then remove the leaf E (case 1). The root is now E, and F keeps only G.

Deleting F from Figure 6 works the same way: go right once to G; G has no left child, so G is the inorder successor. Copy G into F's node and remove the old G, a leaf.

A full deletion example

Start with the tree built in Table 4 and delete 70, then 20, then 40.

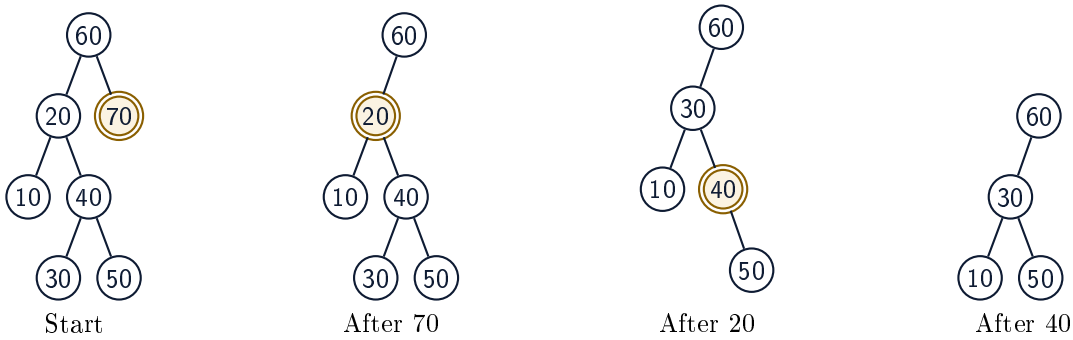


Figure 9: Deleting 70, 20, and 40 in turn; the next node to delete is highlighted with a double border. 70 is a leaf (case 1), so 60 loses its right child. 20 has two children (case 3); its inorder successor is 30, so 30 is copied into 20's node and the leaf 30 is removed. 40 now has one child, 50 (case 2), so 50 moves up to become the right child of 30. The final tree is 60 with left child 30, and 30 has children 10 and 50.

6. BST performance

Search, insert, and delete each follow one path down from the root, doing a constant amount of work per node. So each one is $O(h)$, where h is the height of the tree. The height is everything, and from the bounds in Section 3:

Table 5: Running time of BST operations on a tree with n nodes.

Operation	Best case (minimum height)	Worst case (one long chain)
Search	$O(\log_2 n)$	$O(n)$
Insert	$O(\log_2 n)$	$O(n)$
Delete	$O(\log_2 n)$	$O(n)$
Any traversal (Section 7)	$O(n)$	$O(n)$

To get a feel for the gap: a tree of minimum height holding one million keys has height 20, so a search makes at most 20 comparisons. The same million keys inserted in sorted order make a chain of height one million.

There is a second, practical cost to a tall tree in our code. Every method is recursive and makes one call per level, so a chain of many thousands of nodes can run out of call stack (a `StackOverflowError` in Java) before it ever runs out of memory. Keys inserted in random order usually give a height that is a small multiple of $\log_2 n$, but "usually" is not a guarantee. The guarantee is what balanced trees, next unit, provide.

7. Traversals

A **traversal** visits every node in the tree exactly once. "Visit" means whatever the program needs to do with the item: print it, add it to a total, copy it. There are two families.

- **Depth-first** traversals go "to the leaves" first, finishing an entire subtree before moving on. The three orders differ only in when the node itself is visited compared with its two subtrees:

  - **Preorder**: visit, left, right.
  - **Inorder**: left, visit, right.
  - **Postorder**: left, right, visit.
- **Breadth-first** traversal goes level by level, left to right within each level. It is also called a level-order traversal.

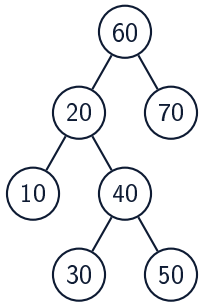


Figure 10: The tree used for the traversal examples (the tree built in Table 4): root 60, with children 20 and 70. 20 has children 10 and 40, and 40 has children 30 and 50.

Table 6: The four traversals of the tree in Figure 10.

Traversal	Order of steps	Visit order
Preorder	visit, left, right	60, 20, 10, 40, 30, 50, 70
Inorder	left, visit, right	10, 20, 30, 40, 50, 60, 70
Postorder	left, right, visit	10, 30, 50, 40, 20, 70, 60
Breadth-first	level by level, left to right	60, 20, 70, 10, 40, 30, 50

Look at the inorder row. An **inorder traversal of a BST visits the keys in sorted order**. That follows straight from the definition: everything in the left subtree is smaller, so it all comes out first, then the node, then everything larger. It is also where the name "inorder successor" comes from: the successor is the node visited right after `tn` in an inorder traversal.

A few common uses: preorder visits a parent before its children, so it is the natural order for copying a tree or saving it to a file so it can be rebuilt with the same shape. Postorder visits the children before the parent, so it is the natural order for destroying a tree (free the children, then the parent). Breadth-first is the natural order for anything that cares about levels, such as finding the node closest to the root that meets some condition.

How would we implement these?

The depth-first traversals are short recursive methods that follow the order of steps in Table 6, with an empty subtree as the base case. Here is preorder; inorder and postorder move the "visit" line.

```
preorder(tn):
    if tn is not null
        visit(tn.item)
        preorder(tn.left)
        preorder(tn.right)
```

Recursion keeps track of where to come back to for us. To traverse without recursion, we have to keep that list of nodes ourselves, and the ADTs from earlier in the course are exactly the right tools.

Inorder with a stack. The stack holds the nodes whose left subtree we are still working through. Go left as far as you can, pushing each node. When you run out of left children, pop a node, visit it, and then do the same thing starting from its right child.

```
inorderWithStack(root):
    cur = root
    stack = empty stack
    while cur is not null or stack is not empty
        while cur is not null          // go left as far as possible
            push cur
            cur = cur.left
        cur = pop                       // leftmost node not yet visited
        visit(cur.item)
        cur = cur.right                 // now handle its right subtree
```

On Figure 10, this pushes 60, 20, 10, then pops and visits 10, pops and visits 20, pushes 40 and 30, pops and visits 30, and so on, producing 10, 20, 30, 40, 50, 60, 70.

Breadth-first with a queue. A queue is first in, first out, so nodes come back out in the same level-by-level order they went in.

```
breadthFirst(root):
    queue = empty queue
    if root is not null
        enqueue root
    while queue is not empty
        tn = dequeue
        visit(tn.item)
        if tn.left  is not null: enqueue tn.left
        if tn.right is not null: enqueue tn.right
```

On Figure 10, the queue starts with 60. Dequeuing 60 adds 20 and 70; dequeuing 20 adds 10 and 40; dequeuing 70 adds nothing; dequeuing 10 adds nothing; dequeuing 40 adds 30 and 50. The visit order is 60, 20, 70, 10, 40, 30, 50.

Your turn. Our `BST` class has an empty `inorder()` method. Writing it, along with preorder, postorder, and breadth-first versions, is left for you. Notice that the public method has no node parameter, so you will want a private helper that does, the same pattern `add` and `insert` use.

8. The code

Java

`Node.java` and `KeyedItem.java` appear in Sections 4 and 5. The exception class is a plain unchecked exception:

Java: BSTException.java

```java
public class BSTException extends RuntimeException {
    public BSTException(String s) {
        super(s);
    }
}
```

The public methods (`add`, `remove`, `search`) each hand the root to a private recursive helper (`insert`, `delete`, `retrieve`). `insert` and `delete` return the new root of the subtree they were given, and the caller stores it back into the link it followed. That one pattern is what lets a recursive method change the shape of the tree without ever holding a reference to the parent.

Java: BST.java

```java
public class BST<T extends KeyedItem<KT>, KT extends Comparable<? super KT>> {

    private Node<T> root;

    public BST() {
        root = null;
    }

    public BST(T item) {
        if(item == null) {
            throw new BSTException("BST Exception: Null Item");
        }
        root = new Node<T>(item);
    }

    public boolean isEmpty() {
        return (root == null);
    }

    public void removeAll() {
        root = null;
    }

    public T peekRoot() {
        if(root == null) {
            throw new BSTException("BST Exception: Empty Tree");
        } else {
            return root.item;
        }
    }

    public void add(T item) {
        if(item == null) {
            throw new BSTException("BST Exception: Null Item");
        }
        root = insert(root, item); // begin inserting process
    }

    private Node<T> insert(Node<T> tn, T item) {
        if(tn == null) { // position found
            tn = new Node<T>(item); // create the new node
        } else {
            Node<T> newSubtree;
            T nodeItem = tn.item;
```

```
            // search for the insertion point
            if(item.getKey().compareTo(nodeItem.getKey()) < 0) {
                // search left
                newSubtree = insert(tn.left, item);
                tn.left = newSubtree;
            } else {
                // search right
                newSubtree = insert(tn.right, item);
                tn.right = newSubtree;
            }
        }
        return tn;
    }

    public void remove(KT searchKey) {
        root = delete(root, searchKey);
    }

    private Node<T> delete(Node<T> tn, KT searchKey) {
        Node<T> newSubtree;

        if(tn == null) {
            throw new BSTException("BST Exception: Item Not Found");
        } else {
            T item = tn.item;
            int cmp = searchKey.compareTo(item.getKey()); // compare once
            if(cmp == 0) {
                tn = deleteNode(tn);
            } else if(cmp < 0) {
                // search left
                newSubtree = delete(tn.left, searchKey);
                tn.left = newSubtree;
            } else {
                // search right
                newSubtree = delete(tn.right, searchKey);
                tn.right = newSubtree;
            }
        }
        return tn;
    }

    private Node<T> deleteNode(Node<T> tn) {
        if(tn.left == null && tn.right == null) {
            // case 1: node is a leaf
            return null;
        } else if (tn.left == null) {
            // case 2a: one child, a right child only
            return tn.right;
        } else if (tn.right == null) {
            // case 2b: one child, a left child only
            return tn.left;
        } else {
            // case 3: two children
            //  note: go right once before it goes left
            T replacement = inorderSuccessor(tn.right);
            tn.item = replacement;
            tn.right = deleteInorderSuccessor(tn.right);
            return tn;
        }
```

```
    }

    private T inorderSuccessor(Node<T> tn) {
        if(tn.left == null) {
            return tn.item; // the replacement item
        } else {
            return inorderSuccessor(tn.left);
        }
    }

    private Node<T> deleteInorderSuccessor(Node<T> tn) {
        if(tn.left == null) {
            return tn.right;
        } else {
            tn.left = deleteInorderSuccessor(tn.left);
            return tn;
        }
    }

    public T search(KT searchKey) {
        // retrieve returns the node that holds the key, or null if the
        // key is not in the tree; .item then returns the item in the node
        Node<T> found = retrieve(root, searchKey);
        if(found == null) {
            throw new BSTException("BST Exception: Item Not Found");
        }
        return found.item;
    }

    private Node<T> retrieve(Node<T> tn, KT searchKey) {
        if(tn == null) {
            return null;
        } else {
            T item = tn.item;
            int cmp = searchKey.compareTo(item.getKey()); // compare once
            if(cmp == 0) {
                return tn;
            } else if(cmp < 0) {
                return retrieve(tn.left, searchKey);
            } else {
                return retrieve(tn.right, searchKey);
            }
        }
    }

    public void inorder() {
        // to be implemented (exercise)
    }

}  // end BST
```

Two details in `deleteNode` are worth pointing out. First, the code has a case 2a and a case 2b because "one child" can be on either side; together they are case 2 from Section 5. (The leaf test in case 1 is not strictly needed, since case 2a would return `tn.right`, which is null for a leaf, but spelling it out matches the three cases.) Second, case 3 walks the right subtree twice: once to find the successor's item, and once to unlink the successor node. `deleteInorderSuccessor` uses the

same "return the new subtree root" pattern as `insert` and `delete`.

C++

The C++ version keeps the same structure and method names. The two real differences are memory and copying. Every node removed from the tree must be freed with `delete`, which happens in `deleteNode` and `deleteInorderSuccessor`, and the destructor frees whatever is left. Copying a tree is disabled, because the default copy would make two trees share the same nodes, and both would try to free them.

C++: BST.h

```
#include <stdexcept>
#include <string>

class BSTException : public std::runtime_error {
public:
    explicit BSTException(const std::string& msg) : std::runtime_error(msg) {}
};

template <typename ItemType, typename KeyType>
class BST {
private:
    struct Node {
        ItemType item;
        Node* left;
        Node* right;
        explicit Node(const ItemType& it) : item(it), left(nullptr), right(nullptr) {}
    };

    Node* root;

    Node* insert(Node* tn, const ItemType& item) {
        if (tn == nullptr) {                    // position found
            tn = new Node(item);                // create the new node
        } else if (item.getKey() < tn->item.getKey()) {
            tn->left = insert(tn->left, item);  // search left
        } else {
            tn->right = insert(tn->right, item); // search right
        }
        return tn;
    }

    Node* deleteNode(Node* tn) {
        if (tn->left == nullptr && tn->right == nullptr) {
            // case 1: node is a leaf
            delete tn;
            return nullptr;
        } else if (tn->left == nullptr) {
            // case 2a: one child, a right child only
            Node* child = tn->right;
            delete tn;
            return child;
        } else if (tn->right == nullptr) {
            // case 2b: one child, a left child only
            Node* child = tn->left;
            delete tn;
```

```
            return child;
        } else {
            // case 3: two children
            //   note: go right once before it goes left
            tn->item = inorderSuccessor(tn->right);
            tn->right = deleteInorderSuccessor(tn->right);
            return tn;
        }
    }

    Node* remove(Node* tn, const KeyType& searchKey) {
        if (tn == nullptr) {
            throw BSTException("BST Exception: Item Not Found");
        }
        if (searchKey == tn->item.getKey()) {
            tn = deleteNode(tn);
        } else if (searchKey < tn->item.getKey()) {
            tn->left = remove(tn->left, searchKey);   // search left
        } else {
            tn->right = remove(tn->right, searchKey); // search right
        }
        return tn;
    }

    const ItemType& inorderSuccessor(Node* tn) const {
        if (tn->left == nullptr) {
            return tn->item;                      // the replacement item
        }
        return inorderSuccessor(tn->left);
    }

    Node* deleteInorderSuccessor(Node* tn) {
        if (tn->left == nullptr) {
            Node* child = tn->right;
            delete tn;                            // C++ must free the node
            return child;
        }
        tn->left = deleteInorderSuccessor(tn->left);
        return tn;
    }

    Node* retrieve(Node* tn, const KeyType& searchKey) const {
        if (tn == nullptr) {
            return nullptr;
        } else if (searchKey == tn->item.getKey()) {
            return tn;
        } else if (searchKey < tn->item.getKey()) {
            return retrieve(tn->left, searchKey);
        } else {
            return retrieve(tn->right, searchKey);
        }
    }

    void destroy(Node* tn) {                      // free both subtrees, then the node
        if (tn != nullptr) {
            destroy(tn->left);
            destroy(tn->right);
            delete tn;
        }
```

```
    }

public:
    BST() : root(nullptr) {}
    ~BST() { destroy(root); }

    // Copying would make two trees share nodes; keep it simple and forbid it.
    BST(const BST&) = delete;
    BST& operator=(const BST&) = delete;

    bool isEmpty() const { return root == nullptr; }

    void removeAll() {
        destroy(root);
        root = nullptr;
    }

    const ItemType& peekRoot() const {
        if (root == nullptr) {
            throw BSTException("BST Exception: Empty Tree");
        }
        return root->item;
    }

    void add(const ItemType& item) { root = insert(root, item); }

    void remove(const KeyType& searchKey) { root = remove(root, searchKey); }

    const ItemType& search(const KeyType& searchKey) const {
        Node* found = retrieve(root, searchKey);
        if (found == nullptr) {
            throw BSTException("BST Exception: Item Not Found");
        }
        return found->item;
    }

    void inorder() const {
        // to be implemented (exercise)
    }
};
```

In Java, `removeAll` only sets `root` to null and lets the garbage collector reclaim the nodes. In C++, nothing reclaims them for you, so `removeAll` calls `destroy` first. Without that call the method would compile and run, and quietly leak every node in the tree.

9. Things that trip people up

- **Forgetting to store the returned subtree.** Writing `insert(tn.left, item);` without `tn.left =` in front builds the new node and then throws it away. The tree never changes, and nothing reports an error.
- **Checking only the children.** A BST requires every key in the whole left subtree to be smaller, not just the left child. A tree can pass the parent and child test at every node and still not be a BST.
- **Looking for the successor in the wrong place.** The inorder successor of a node with two

children is in its *right* subtree: go right once, then all the way left. It is not the node's parent, and it is not simply its right child (unless that child has no left child).

- **Relinking in case 3.** With two children, we copy the successor's item into the node and remove the successor. Trying to unlink the node itself leaves two subtrees and only one link to attach them to.
- **Mixing height conventions.** Counting nodes and counting edges give answers that differ by one. Check which one a source uses.
- **Assuming a BST is fast.** A BST is only as fast as it is short. Sorted input makes it a linked list.